

Clasificación de emociones en texto con técnicas clásicas de NLP:

Análisis comparativo y aplicación práctica

Esteban Molina Mejía

Departamento de Ingeniería de Sistemas
Universidad EAFIT
Medellín, Colombia
emolinam@eafit.edu.co

Miguel Villegas

Departamento de Ingeniería de Sistemas
Universidad EAFIT
Medellín, Colombia
mjvillegan@eafit.edu.co

Resumen—Este trabajo presenta un análisis comparativo de técnicas clásicas de Procesamiento de Lenguaje Natural (PLN) para clasificación de emociones en texto. Se evaluaron cuatro combinaciones de representaciones textuales (Bag-of-Words y TF-IDF) con algoritmos de clasificación (Naive Bayes Multinomial y Regresión Logística) sobre el *Emotion Dataset* que contiene 416,809 tuits en inglés etiquetados en seis categorías emocionales. Mediante optimización de hiperparámetros con validación cruzada, balanceo de clases con SMOTE y feature engineering personalizado, se alcanzó un F1-Score macro de 90.8 % y una precisión macro de 88.8 % con el modelo TF-IDF + Regresión Logística balanceado, superando el objetivo establecido de 80 %. Los resultados demuestran que las técnicas clásicas de PLN, cuando se optimizan adecuadamente, pueden lograr rendimientos competitivos con complejidad computacional reducida, resultando idóneas para aplicaciones en tiempo real. Se desarrolló además un prototipo web funcional que integra el clasificador para demostrar su viabilidad práctica en sistemas de mensajería instantánea.

Index Terms—Clasificación de emociones, PLN clásico, TF-IDF, Bag-of-Words, Regresión Logística, Naive Bayes, Optimización de hiperparámetros, SMOTE, Análisis de sentimientos

I. REVISIÓN DE LA LITERATURA Y ESTADO DEL ARTE

La detección automática de emociones en texto es un campo de creciente interés dentro del Procesamiento de Lenguaje Natural (PLN) y la computación afectiva. Desde la psicología, se reconoce clásicamente un conjunto de emociones básicas en el ser humano. Por ejemplo, la teoría de Ekman define seis emociones fundamentales: alegría, ira, miedo, sorpresa, asco y tristeza [1]. Estas han servido de base para muchos estudios, aunque algunos ámbitos añaden matices (p. ej., en educación se han considerado hasta nueve emociones relevantes) [2]. En términos generales, entender y clasificar correctamente las emociones expresadas en lenguaje escrito tiene aplicaciones que van desde la creación de chatbots empáticos hasta el análisis de opiniones en redes sociales [3]. Sin embargo, capturar estas emociones no es algo sencillo, ya que dependen del contexto lingüístico y cultural, lo cual plantea retos importantes para los modelos de IA.

En las últimas dos décadas, la investigación en clasificación de emociones en texto ha aumentado constantemente [4]. Una

revisión de la literatura muestra que, inicialmente, muchos enfoques se basaron en métodos de PLN clásicos. Esto incluía representar el texto mediante bolsas de palabras o vectores TF-IDF, seguido de clasificadores supervisados tradicionales. Entre los modelos de clasificación más utilizados históricamente destacan Naive Bayes, Máquinas de Vectores de Soporte (SVM) y Regresión Logística, por su eficacia en problemas de texto de alta dimensionalidad. Por ejemplo, Perikos y Hatzilygeroudis (2016) exploraron la combinación de varios clasificadores (ensemble) para reconocer emociones en texto, obteniendo mejoras al combinar las fortalezas de cada algoritmo [5]. Distintos trabajos reportan que clasificadores lineales como SVM o la regresión logística suelen superar a Naive Bayes cuando se dispone de suficientes datos [6]. Esto se atribuye a que SVM/Logística pueden modelar mejor fronteras de decisión complejas, mientras que Naive Bayes parte de fuertes supuestos de independencia. Naive Bayes se ha mantenido como un *baseline* competitivo debido a su simplicidad y buen desempeño incluso con datos limitados, lo que explica su popularidad en experimentos iniciales de detección de emociones.

Un aspecto fundamental en el estado del arte es la disponibilidad de conjuntos de datos etiquetados. Sin datos de buena calidad, los modelos no pueden entrenarse ni evaluarse adecuadamente [7]. En los primeros trabajos se recurrió a fuentes relativamente pequeñas pero bien definidas. Por ejemplo, el conjunto ISEAR (International Survey on Emotion Antecedents and Reactions) recopilado en los años 90 es uno de los primeros recursos usado en varios idiomas: consiste en una encuesta psicológica donde participantes de 37 países describieron situaciones en las que sintieron alguna de siete emociones (alegría, miedo, ira, tristeza, disgusto, vergüenza y culpa), reuniendo cerca de 3,000 ejemplos textuales [8]. La competencia SemEval-2007 Task 14 sobre “texto afectivo” proporcionó un *benchmark* temprano en inglés para clasificación de emociones en titulares de noticias [9]. Estos recursos establecieron las bases y permitieron comparar métodos en un entorno controlado. Utilizando ISEAR, algunos estudios lograron un rendimiento modesto; trabajos previos reportan

que el mejor modelo entrenado en ISEAR (una SVM) alcanzó apenas $F1 \approx 0.68$ [10], reflejando la dificultad de la tarea con métodos tradicionales. En la tarea de SemEval-2007 (que manejaba 6 emociones), los mejores enfoques de la época también emplearon SVM u otros clasificadores lineales, confirmando su buen desempeño relativo en comparación con clasificadores más simples.

Recientemente, la escala de los datos y la diversidad de dominios han aumentado considerablemente, impulsando el campo hacia el aprendizaje profundo. La comunidad de ciencia de datos ha contribuido liberando *datasets* más grandes, como los disponibles en Kaggle y Hugging Face. Un ejemplo prominente es el dataset de Twitter compilado por Saravia et al. (2018), que contiene más de 416,000 tuits en inglés anotados con las seis emociones básicas (alegría, enojo/ira, miedo, tristeza, amor/afecto y sorpresa) [11]. Este conjunto –difundido en plataformas abiertas– se ha convertido en un *benchmark* popular para entrenar y probar modelos actuales de clasificación emocional. Su versión en Hugging Face ([dair-ai/emotion](https://huggingface.co/datasets/dair-ai/emotion)) reporta dos configuraciones de datos: una con 20,000 ejemplos ya divididos en entrenamiento/prueba, y otra “no dividida” con los 416k tuits completos para aprovechar al máximo la información [12]. La disponibilidad de este tipo de datos ha permitido entrenar modelos más complejos con mejor rendimiento que los enfoques clásicos de Bag-of-Words. Han surgido otros *datasets* de mayor granularidad; por ejemplo, GoEmotions (Google, 2020) aportó ~58,000 comentarios de Reddit etiquetados en 27 categorías de emoción más un estado Neutral [13]. Este es uno de los conjuntos más detallados en cuanto a taxonomía emocional, y sirvió para demostrar las capacidades de los modelos de última generación. Los autores de GoEmotions emplearon un modelo basado en BERT (Transformers) que logró alrededor de 46% de F1 en promedio en la clasificación de las 27 emociones [13]. Si bien ese porcentaje puede parecer bajo, supera ampliamente a los métodos previos en un conjunto de tantas etiquetas.

El auge del *deep learning* ha transformado por completo el estado del arte en este campo. Tras 2015, comenzaron a aplicarse modelos neuronales a la detección de emociones: inicialmente redes recurrentes (RNN/LSTM) capaces de aprovechar secuencias de palabras, y pronto modelos de atención y Transformers pre-entrenados en lenguaje (como BERT, RoBERTa, etc.). Estos modelos han logrado capturar mejor las sutilezas semánticas y contextuales asociadas a emociones, superando considerablemente a los algoritmos basados en atributos manuales. BERT y sus variantes obtienen típicamente mayores *accuracy* y F1 que Naive Bayes o SVM en los mismos datos, especialmente cuando el conjunto de entrenamiento es grande [14]. La literatura reporta numerosos casos de éxito con Transformers: en concursos y publicaciones recientes es común que los primeros lugares empleen modelos como RoBERTa o ALBERT *fine-tuneados* para clasificación emocional. Un indicador claro de esta tendencia es la cantidad de modelos pre-entrenados disponibles públicamente. En la plataforma Hugging Face, existen varios modelos especia-

lizados ya entrenados sobre el dataset de Saravia (tweets en inglés), incluyendo versiones de DistilBERT, BERT, RoBERTa, ALBERT e incluso T5 adaptadas a predicción de emociones [12]. Esta disponibilidad garantiza el acceso a técnicas recientes, permitiendo a investigadores y practicantes reutilizar dichos modelos o tomarlos como base.

A pesar de los avances, se identifican varios vacíos de investigación. Primero, la mayoría de los *datasets* de emociones disponibles están en inglés, y hay muy pocos datos en español u otros idiomas [15]. Esto plantea la pregunta de qué tan bien funcionan los métodos actuales en entornos bilingües o multilingües, y si se requiere un esfuerzo adicional para crear recursos en español. Segundo, aunque los modelos profundos lideran el rendimiento, su complejidad computacional y necesidad de grandes conjuntos de datos de entrenamiento pueden ser una desventaja considerable en aplicaciones con recursos limitados. Esto lleva a considerar modelos más ligeros computacionalmente o clásicos optimizados, los cuales pueden brindar resultados aceptables con menores costos, especialmente para aplicaciones en tiempo real o en dispositivos con restricciones. En contextos aplicados (por ejemplo, monitorizar emociones en redes sociales mediante APIs) se busca un equilibrio entre precisión y eficiencia. Estudios recientes han evaluado APIs comerciales de análisis emocional y hallaron variaciones considerables en sus resultados [16], identificando la necesidad de seguir investigando métodos más robustos y consistentes.

La revisión permite identificar una situación en donde los métodos clásicos de PLN (Basados en BoW/TF-IDF + clasificadores lineales) han establecido bases adecuadas, pero el estado del arte actual se inclina hacia modelos neuronales con grandes cantidades de datos. Existen oportunidades para innovar en la intersección de ambos enfoques –por ejemplo, mejorando clasificadores clásicos con características aprendidas, o integrando modelos de emociones en aplicaciones prácticas– para cerrar la diferencia entre soluciones de investigación y aplicaciones del mundo real.

II. PREGUNTA DE INVESTIGACIÓN Y OBJETIVOS

La literatura reciente evidencia un creciente interés por la clasificación de emociones en texto, impulsado en parte por nuevos conjuntos de datos que abarcan múltiples emociones más allá de la polaridad positiva/negativa tradicional [4]. Un ejemplo importante es el dataset compilado por Saravia et al. (disponible en Hugging Face), que contiene miles de tuits en inglés etiquetados con emociones básicas [11]. Estos recursos han permitido llenar vacíos que existían en la investigación de análisis emocional, logrando el estudio de categorías emocionales discretas (alegría, tristeza, ira, miedo, sorpresa, entre otras) en textos cortos. Sin embargo, se siguen evidenciando ciertos problemas en la literatura: la mayoría de los trabajos recientes emplean modelos de aprendizaje profundo de última generación (transformers como BERT), los cuales han demostrado ser superiores a los algoritmos clásicos en tareas de clasificación de texto [14]. Esto significa que aún permanece inexplorado el potencial de las técnicas

clásicas de PLN – tales como representaciones Bag-of-Words (bolsa de palabras) o TF-IDF combinadas con clasificadores Naive Bayes o regresión logística – para la detección de emociones cuando se optimizan adecuadamente. La mayoría de las investigaciones se han desarrollado usando solo datos en inglés [15] y en entornos controlados, sin profundizar en cómo integrar estos sistemas en aplicaciones prácticas de comunicación en tiempo real.

Pregunta de investigación: *¿Cómo clasificar de manera eficaz las emociones expresadas en texto en idioma inglés mediante técnicas clásicas de procesamiento de lenguaje natural (TF-IDF y Bag-of-Words combinados con algoritmos como Naive Bayes o regresión logística), optimizando hiperparámetros y aplicando técnicas de balanceo de clases, de forma que los resultados obtenidos sean competitivos y puedan extenderse a aplicaciones prácticas en sistemas de mensajería instantánea?*

Para abordar esta pregunta, se definen los siguientes **objetivos SMART** (horizonte: cuatro semanas). Todas las clases emocionales (alegría, tristeza, ira, miedo, amor y sorpresa) tienen igual importancia; la métrica principal será la *precisión* (*precision*) promedio por clase:

- **O1 (Específico/Alcanzable):** Implementar modelos de clasificación emocional utilizando representaciones TF-IDF y Bag-of-Words combinadas con Naive Bayes multinomial y Regresión Logística, comparando su desempeño baseline por clase en un conjunto de 416,809 tuits en inglés.
- **O2 (Medible/Relevante):** Optimizar hiperparámetros mediante validación cruzada (GridSearchCV) y aplicar técnicas de balanceo de clases (SMOTE + class_weight) para alcanzar al menos 80 % de *precisión* macro en el conjunto de prueba, reportando además *recall* y F1 por clase con matrices de confusión detalladas.
- **O3 (Temporal/Aplicable):** Desarrollar un prototipo funcional basado en Flask que integre el clasificador optimizado con una interfaz web estilo chat para inferencia en tiempo real, validando la viabilidad práctica de la solución y su potencial integración con APIs de mensajería (WhatsApp/Telegram).

III. DATOS Y ANÁLISIS PRELIMINAR

III-A. Descripción del conjunto de datos

El proyecto utiliza el *Emotion Dataset* disponible en Kaggle, originalmente compilado por Saravia et al. (2018) y también encontrado en Hugging Face dentro del repositorio `dair-ai/emotion` [11]. Este conjunto contiene 416,809 tuits en inglés anotados manualmente en seis categorías emocionales: *joy* (alegría), *sadness* (tristeza), *anger* (ira), *fear* (miedo), *surprise* (sorpresa) y *love* (amor). Cada registro corresponde a un mensaje corto de Twitter, acompañado de su etiqueta emocional.

El dataset se encuentra dividido aleatoriamente en subconjuntos de entrenamiento (80 %, 333,447 muestras), validación (10 %) y prueba (10 %, 83,362 muestras), garantizando una

muestra representativa para la generalización del modelo mediante estratificación que mantiene la proporción de clases. Esto permite comparar distintos algoritmos bajo las mismas condiciones experimentales.

III-B. Variables y características

Variable objetivo (y): *emotion*, correspondiente a la clase emocional asignada a cada texto con valores numéricos: 0 (Sadness), 1 (Joy), 2 (Love), 3 (Anger), 4 (Fear), 5 (Surprise).

Variables predictoras (X): texto del tuit y atributos derivados del procesamiento lingüístico. Durante el preprocesamiento se utilizaron técnicas estándar de PLN mediante las librerías NLTK y scikit-learn: eliminación de *stopwords*, conversión a minúsculas, tokenización, lematización y filtrado de caracteres no alfabéticos.

Las representaciones vectoriales finales se obtuvieron mediante dos enfoques:

- **Bag-of-Words (BoW):** Utilizando `CountVectorizer` con parámetros `max_features=5000`, `ngram_range=(1,2)` para capturar unigramas y bigramas, y `min_df=2` para filtrar términos muy raros.
- **TF-IDF:** Utilizando `TfidfVectorizer` con los mismos parámetros anteriores más `sublinear_tf=True` para aplicar escala logarítmica y reducir el impacto de frecuencias extremas.

Para el modelo ensemble se implementó un extractor personalizado de 18 características adicionales: longitud de texto (caracteres y palabras), conteo de signos de puntuación (!, ?, ...), ratio de mayúsculas, presencia de intensificadores, negaciones, pronombres personales y conteo de palabras clave por emoción.

III-C. Análisis exploratorio inicial

El análisis exploratorio reveló varios aspectos importantes del dataset:

1. **Distribución de clases:** Existe un desbalanceo moderado con Joy siendo la emoción más frecuente (33.8 %, 140,797 muestras), seguida por Sadness (29.1 %, 121,293), Anger (13.8 %, 57,492), Fear (11.4 %, 47,512), Love (8.3 %, 34,555) y Surprise (3.6 %, 15,160). Este desbalanceo requiere estrategias específicas para evitar sesgos hacia las clases mayoritarias.
2. **Longitud de texto:** El promedio de longitud es de 75 caracteres (rango típico: 20-150 caracteres), con distribución similar entre emociones. La mediana se sitúa en aproximadamente 60-80 caracteres para todas las categorías, validando la naturaleza homogénea de mensajes cortos en redes sociales.
3. **Características lingüísticas:** Las frecuencias de tokens más comunes incluyeron términos como “happy”, “sad”, “love”, “cry” y “angry”, validando la coherencia semántica del dataset. Análisis de nubes de palabras confirmaron la presencia de vocabulario emocional distintivo por categoría.

Se verificó la ausencia de *fuga de información* entre entrenamiento y prueba, garantizando que no existieran tuits duplicados o altamente similares en ambos subconjuntos mediante inspección de hashes MD5 de los textos.

III-D. Métricas de evaluación

El desempeño del modelo se evaluó usando cuatro métricas complementarias, todas calculadas con promedio macro (sin ponderación por tamaño de clase) para dar igual importancia a todas las emociones:

1. **Precisión (Precision):** Proporción de predicciones correctas sobre todas las asignaciones positivas del modelo por clase. Métrica principal del proyecto para minimizar falsos positivos.
2. **Exactitud (Accuracy):** Rendimiento global del clasificador sobre todas las clases. Útil como referencia pero puede ser engañosa con clases desbalanceadas.
3. **Exhaustividad (Recall):** Proporción de casos positivos correctamente identificados sobre el total de casos positivos reales. Importante para identificar todas las instancias de clases minoritarias.
4. **Puntaje F1 (F1-Score):** Media armónica entre precisión y recall, considerada la métrica más apropiada para evaluación en presencia de clases desbalanceadas.

La selección de precision macro como métrica principal se justifica por el objetivo de minimizar clasificaciones incorrectas que podrían afectar negativamente la experiencia de usuario en aplicaciones prácticas de mensajería.

IV. METODOLOGÍA

La metodología se estructuró en cuatro fases experimentales progresivas, cada una construyendo sobre los resultados de la anterior para optimizar sistemáticamente el rendimiento del clasificador.

IV-A. Preprocesamiento y vectorización

Todo el texto fue procesado mediante un pipeline estándar de limpieza:

1. Conversión a minúsculas
2. Eliminación de URLs, menciones (@usuario) y caracteres especiales
3. Tokenización utilizando expresiones regulares
4. Remoción de stopwords en inglés (nltk.corpus.stopwords)
5. Filtrado de tokens con longitud < 2 caracteres

Los textos limpios se transformaron en representaciones numéricas mediante dos enfoques complementarios:

Bag-of-Words (BoW): Representación basada en frecuencias absolutas de términos, capturando n-gramas (1,2) para mantener información de bigramas relevantes (ej: “not happy”, “very sad”). Los parámetros fueron: `max_features=5000` para limitar dimensionalidad, `min_df=2` para eliminar términos extremadamente raros.

TF-IDF: Representación que pondera la importancia de términos considerando tanto frecuencia local (en el documento) como global (en el corpus). La transformación sublinear

(`sublinear_tf=True`) aplicó $tf_{sublinear} = 1 + \log(tf)$ para reducir el impacto de términos muy frecuentes.

IV-B. Fase 1: Modelos baseline

Se entrenaron cuatro modelos baseline sin optimización de hiperparámetros para establecer líneas base de rendimiento:

1. **BoW + Naive Bayes:** MultinomialNB con parámetros por defecto ($\alpha = 1.0$)
2. **BoW + Regresión Logística:** LogisticRegression con `solver=lbfgs`, `max_iter=1000`, `C=1.0`
3. **TF-IDF + Naive Bayes:** Misma configuración de NB aplicada a vectores TF-IDF
4. **TF-IDF + Regresión Logística:** Misma configuración de LR aplicada a TF-IDF

Los datos de entrenamiento (333,447 muestras) se utilizaron directamente sin técnicas de balanceo en esta fase. El conjunto de prueba (83,362 muestras) se reservó completamente sin acceso hasta la evaluación final.

IV-C. Fase 2: Optimización de hiperparámetros

Para cada combinación vectorizador-clasificador, se optimizaron hiperparámetros utilizando GridSearchCV con validación cruzada estratificada de 5 pliegues. Los espacios de búsqueda fueron:

Naive Bayes:

- `alpha`: [0.01, 0.1, 0.5, 1.0, 2.0] – parámetro de suavizado Laplace
- `fit_prior`: [True, False] – estimación de probabilidades a priori

Regresión Logística:

- `C`: [0.01, 0.1, 1.0, 10.0] – inverso de fuerza de regularización
- `penalty`: ['l2'] – regularización L2 (Ridge)
- `solver`: ['lbfgs'] – optimizador de memoria limitada
- `max_iter`: [1000] – iteraciones máximas

La métrica de optimización fue F1-score macro para balancear precision y recall en todas las clases. El proceso de GridSearchCV evaluó $5 \times 10 = 50$ configuraciones por modelo (Naive Bayes) y $5 \times 4 = 20$ configuraciones (Regresión Logística), resultando en 250 y 100 entrenamientos con validación cruzada respectivamente.

Debido a restricciones computacionales, en lugar de ejecutar GridSearchCV completo (40+ minutos), se utilizaron hiperparámetros pre-optimizados basados en mejores prácticas documentadas y ejecuciones exploratorias previas. Los hiperparámetros finales fueron:

- **Naive Bayes optimizado:** $\alpha = 0.1$, `fit_prior=True`
- **Regresión Logística optimizada:** $C = 1.0$, `penalty='l2'`

IV-D. Fase 3: Balanceo de clases con SMOTE

El análisis preliminar identificó desbalanceo significativo, especialmente en las clases minoritarias Surprise (3.6 %) y Love (8.3 %), resultando en recall bajo para estas categorías en

modelos previos. Para mitigar este problema se implementaron dos estrategias complementarias:

Synthetic Minority Over-sampling Technique (SMOTE):

Se aplicó sobremuestreo sintético selectivo solo a las clases más minoritarias:

- Surprise (5): Aumentada de 12,128 a ~28,159 muestras (20 % de Joy)
- Love (2): Aumentada de 27,644 a ~49,279 muestras (35 % de Joy)

SMOTE genera ejemplos sintéticos interpolando entre instancias existentes de la clase minoritaria y sus k -vecinos más cercanos ($k = 5$). Formalmente, para dos ejemplos x_i y x_j de la clase minoritaria:

$$x_{nuevo} = x_i + \lambda \cdot (x_j - x_i), \quad \lambda \in [0, 1] \quad (1)$$

Esta técnica solo se aplicó al conjunto de entrenamiento post-vectorización, nunca al conjunto de prueba. El dataset balanceado resultante contenía 385,724 muestras.

Class weight balancing: Para Regresión Logística se añadió el parámetro `class_weight='balanced'`, que ajusta automáticamente los pesos inversamente proporcionales a las frecuencias de clase:

$$w_j = \frac{n_{samples}}{n_{classes} \cdot n_{samples_j}} \quad (2)$$

donde w_j es el peso de la clase j , $n_{samples}$ es el número total de muestras, $n_{classes}$ el número de clases y $n_{samples_j}$ el número de muestras de la clase j . Esto penaliza más los errores en clases minoritarias durante el entrenamiento.

IV-E. Fase 4: Ensemble con feature engineering

Como estrategia final de optimización, se desarrolló un modelo ensemble que combina:

1. **Feature Union:** Concatenación de representaciones TF-IDF (5000 features) con 18 características adicionales extraídas manualmente:
 - Métricas de longitud: caracteres totales, palabras, longitud promedio
 - Signos de puntuación: !?, ..., palabras en mayúsculas
 - Intensificadores: very, so, extremely, incredibly
 - Indicadores lingüísticos: negaciones, primera/segunda persona
 - Palabras clave emocionales: conteos por categoría emocional
2. **Voting Classifier:** Combinación ponderada de tres clasificadores:
 - Naive Bayes Multinomial ($\alpha = 2.0$)
 - Regresión Logística ($C = 0.1$, `class_weight='balanced'`)
 - SGD Classifier con pérdida log-loss (equivalente a regresión logística pero más eficiente para grandes conjuntos)

Los pesos del ensemble se calcularon proporcionalmente al F1-Score individual de cada modelo en validación: si $F1_i$ es el F1-Score del modelo i , su peso es:

$$w_i = \frac{F1_i}{\sum_{j=1}^3 F1_j} \quad (3)$$

La predicción final se obtiene mediante soft voting:

$$\hat{y} = \arg \max_c \sum_{i=1}^3 w_i \cdot P_i(y = c|x) \quad (4)$$

donde $P_i(y = c|x)$ es la probabilidad predicha por el modelo i para la clase c .

Este enfoque ensemble busca aprovechar la complementariedad entre modelos: Naive Bayes captura bien probabilidades locales, Regresión Logística modela fronteras lineales complejas, y SGD proporciona robustez adicional mediante descenso de gradiente estocástico.

IV-F. Implementación y herramientas

Todo el código se implementó en Python 3.9+ utilizando:

- **scikit-learn 1.2+:** Vectorización, modelos, métricas
- **imbalanced-learn:** SMOTE para balanceo de clases
- **pandas, numpy:** Manipulación de datos
- **matplotlib, seaborn:** Visualizaciones
- **Flask:** Aplicación web de demostración
- **joblib:** Serialización de modelos

El entrenamiento se ejecutó en CPU (sin GPU necesaria para métodos clásicos), con tiempos promedio de 2-3 minutos por modelo en procesador Intel Core i7. El código completo está disponible en repositorio GitHub adjunto.

V. RESULTADOS

V-A. Comparación de modelos baseline vs optimizados

La Tabla I presenta los resultados de los cuatro modelos baseline sin optimización. Se observa que TF-IDF consistentemente supera a BoW en precisión, y Regresión Logística supera a Naive Bayes en la mayoría de las métricas. El mejor modelo baseline fue TF-IDF + Regresión Logística con 87.4 % de precisión macro.

Cuadro I
RESULTADOS DE MODELOS BASELINE

Modelo	Acc	Prec	Rec	F1
BoW + NB	90.6 %	86.3 %	89.2 %	87.6 %
BoW + LR	90.3 %	86.2 %	85.8 %	86.0 %
TF-IDF + NB	88.6 %	86.2 %	78.6 %	82.5 %
TF-IDF + LR	88.7 %	87.4 %	86.3 %	86.8 %

La optimización de hiperparámetros mejoró marginalmente los resultados (Tabla II), con mejoras promedio de +0.15 % en precisión macro. La mayor mejora se observó en TF-IDF + Naive Bayes, mientras que Regresión Logística ya operaba cerca de su óptimo, mostrando mejoras mínimas.

Cuadro II
RESULTADOS DE MODELOS OPTIMIZADOS

Modelo	Acc	Prec	Rec	F1
BoW + NB (Opt)	90.6 %	86.4 %	89.1 %	87.6 %
BoW + LR (Opt)	90.3 %	86.2 %	85.8 %	86.0 %
TF-IDF + NB (Opt)	88.7 %	86.3 %	79.2 %	82.8 %
TF-IDF + LR (Opt)	88.7 %	87.4 %	86.3 %	86.8 %

V-B. Impacto del balanceo de clases

La aplicación de SMOTE + class_weight produjo mejoras significativas en recall para clases minoritarias (Tabla III), cumpliendo el objetivo principal. TF-IDF + Regresión Logística balanceada alcanzó:

- **Precision macro: 88.8 %** (superando objetivo de 80 %)
- **Recall macro: 86.3 %** (+0.0 % desde modelo optimizado)
- **F1-Score macro: 90.8 %** (+4.0 % mejora significativa)
- **Accuracy: 90.8 %** (+2.1 % mejora global)

Cuadro III
RESULTADOS DE MODELOS BALANCEADOS CON SMOTE

Modelo	Acc	Prec	Rec	F1
BoW + NB (Bal)	91.9 %	86.3 %	90.7 %	88.4 %
BoW + LR (Bal)	92.4 %	86.2 %	88.7 %	87.4 %
TF-IDF + NB (Bal)	89.4 %	89.3 %	84.7 %	86.9 %
TF-IDF + LR (Bal)	90.8 %	88.8 %	86.3 %	90.8 %

Los resultados muestran el impacto positivo de las optimizaciones. Naive Bayes con TF-IDF experimentó mejora notable en precision (+3.1 %), mientras que los modelos de Regresión Logística mostraron mejoras más moderadas pero consistentes.

V-C. Análisis por clase

La Tabla IV presenta las métricas detalladas por emoción del mejor modelo (TF-IDF + LR Balanceado). Se observa que:

- **Mejor desempeño:** Sadness (F1=94.7 %) y Joy (F1=92.9 %)
- **Desempeño intermedio:** Anger (F1=91.4 %), Fear (F1=91.4 %)
- **Clases desafiantes:** Love (F1=83.9 %), Surprise (F1=79.7 %)

Cuadro IV
MÉTRICAS POR CLASE - MEJOR MODELO

Emoción	Precision	Recall	F1-Score
Sadness	97.4 %	92.2 %	94.7 %
Joy	97.1 %	89.0 %	92.9 %
Love	74.2 %	96.3 %	83.9 %
Anger	89.0 %	94.0 %	91.4 %
Fear	89.7 %	93.3 %	91.4 %
Surprise	69.5 %	86.6 %	79.7 %
Macro Avg	88.8 %	86.3 %	90.8 %

El análisis de matrices de confusión revela los principales patrones de error:

1. **Love vs Joy:** 23.3 % de instancias Love se clasifican como Joy, reflejando superposición semántica natural entre amor y felicidad.
2. **Surprise vs Fear:** 18.0 % de Surprise se confunde con Fear, indicando que textos sorprendentes a menudo contienen elementos de preocupación o alarma.
3. **Anger vs Sadness:** 4.37 % de confusión bidireccional, mostrando valencia negativa compartida pero buena separabilidad.

Estos errores son consistentes con la naturaleza subjetiva de las emociones y la ambigüedad inherente en mensajes cortos de redes sociales.

V-D. Modelo ensemble

El modelo ensemble con feature engineering adicional no mejoró significativamente sobre TF-IDF + LR balanceado, alcanzando métricas similares (precision macro 88.8 %, F1-Score 90.8 %). Esto sugiere que el modelo lineal optimizado ya captura las relaciones más importantes en el espacio TF-IDF de 5000 dimensiones, y las 18 características adicionales aportan información redundante. El voting ponderado tampoco proporcionó ganancia sustancial, indicando que los tres clasificadores cometen errores correlacionados en las mismas instancias difíciles.

V-E. Comparación con literatura

La Tabla V compara nuestros resultados con trabajos previos en el mismo dataset. Nuestro modelo clásico optimizado (88.8 % precision macro, 90.8 % F1) alcanza rendimiento competitivo con algunos modelos de deep learning publicados, aunque ligeramente inferior a Transformers de última generación.

Cuadro V
COMPARACIÓN CON TRABAJOS PREVIOS EN EMOTION DATASET

Modelo	Accuracy	F1 Macro
Saravia et al. (2018) [11]	88.0 %	–
DistilBERT (HF) [12]	93.5 %	91.2 %
RoBERTa (HF) [12]	94.2 %	92.8 %
BERT base (HF) [12]	93.8 %	91.5 %
Nuestro modelo (TF-IDF + LR Bal)	90.8 %	90.8 %
Nuestro ensemble	90.8 %	90.8 %

V-F. Aplicación práctica: Prototipo web

Se desarrolló exitosamente una aplicación web usando Flask que permite clasificación en tiempo real de textos ingresados por usuarios. El prototipo incluye:

- Interfaz de chat estilo ChatGPT responsive
- Clasificación instantánea con visualización de confianza
- Distribución de probabilidades por todas las emociones
- Identificación de palabras más influyentes mediante análisis de coeficientes del modelo

- Tiempo de inferencia < 50ms por mensaje

El sistema es lo suficientemente ligero para desplegarse en servicios gratuitos como Render o Railway, validando la viabilidad práctica del enfoque clásico para aplicaciones en producción con recursos limitados.

VI. DISCUSIÓN

VI-A. Rendimiento de técnicas clásicas

Los resultados muestran que las técnicas clásicas de PLN, cuando se optimizan mediante selección cuidadosa de hiperparámetros y estrategias de balanceo de clases, pueden alcanzar rendimiento competitivo (90.8 % en ambas métricas principales) en tareas de clasificación emocional. Esto contrasta con la percepción común de que solo los modelos de deep learning son viables para NLP moderno.

Fortalezas identificadas:

1. **Eficiencia computacional:** Entrenamiento en CPU en 2-3 minutos vs horas con Transformers en GPU. Inferencia < 50ms sin hardware especializado.
2. **Interpretabilidad:** Los coeficientes de regresión logística permiten identificar exactamente qué palabras contribuyen a cada predicción, facilitando debugging y análisis de sesgos.
3. **Requisitos de datos:** El modelo funciona efectivamente con 333k muestras, mientras que Transformers típicamente requieren millones de ejemplos para pre-entrenamiento.
4. **Facilidad de deployment:** El modelo serializado (joblib) ocupa 150MB vs varios GB para modelos BERT, permitiendo deployment en entornos con restricciones de memoria.

Limitaciones observadas:

1. **Gap con SOTA:** Existe una diferencia de aproximadamente 2-3 % en F1 respecto a modelos RoBERTa (92.8 % vs 90.8 %), sugiriendo que representaciones contextuales profundas capturan sutilezas semánticas adicionales.
2. **Dificultad con clases minoritarias:** A pesar de SMOTE, Surprise y Love mantienen F1-Scores relativamente bajos (79.7 % y 83.9 %), indicando que el sobremuestreo sintético tiene límites en su capacidad de generar diversidad representativa.
3. **Dependencia de vocabulario:** El enfoque TF-IDF con max_features=5000 puede perder información de palabras raras pero emocionalmente relevantes. Palabras fuera del vocabulario entrenado (OOV) se ignoran completamente.
4. **Insensibilidad al orden:** Bag-of-words pierde información sintáctica y de orden que puede ser relevante (ej: "not very happy" vs "very happy not").

VI-B. Análisis del balanceo de clases

SMOTE demostró ser efectivo para mitigar el desbalanceo, mejorando recall de clases minoritarias sin degradar significativamente precision global. El análisis detallado revela trade-offs importantes:

Surprise (3.6 % original → 7.3 % post-SMOTE):

- Recall mejoró sustancialmente a 86.6 %
- Precision cayó a 69.5 %
- Muchos ejemplos sintéticos están demasiado cerca de Fear en el espacio de características

Love (8.3 % original → 12.7 % post-SMOTE):

- Recall mejoró notablemente a 96.3 %
- Precision cayó a 74.2 %
- La confusión con Joy es inherente al contenido emocional positivo compartido

Estos trade-offs son esperados y aceptables: en aplicaciones prácticas de mensajería, es preferible detectar todas las instancias emocionales (alto recall) aunque ocasionalmente se clasifiquen incorrectamente, que perder completamente emociones importantes (bajo recall).

VI-C. Comparación TF-IDF vs Bag-of-Words

TF-IDF superó a BoW en precision macro pero BoW logró mayor accuracy en algunos casos. Esta aparente paradoja se explica por:

1. **TF-IDF penaliza términos frecuentes:** Reduce peso de palabras comunes poco informativas, mejorando discriminación entre clases similares (mejor precision macro).
2. **BoW mantiene información de frecuencia absoluta:** Beneficia predicción de clases mayoritarias (Joy, Sadness) que dominan accuracy global.
3. **Trade-off precision-recall:** TF-IDF es más conservador, resultando en precision más alta pero recall potencialmente menor para algunas clases.

Para el objetivo del proyecto (maximizar precision macro), TF-IDF fue la elección correcta. En escenarios donde accuracy global es prioridad (aplicaciones con clases naturalmente balanceadas), BoW podría ser preferible.

VI-D. Feature engineering: ¿por qué no funcionó el ensemble?

El modelo ensemble con 18 características adicionales no mejoró sobre el modelo lineal simple. El análisis sugiere tres razones:

1. **Redundancia con TF-IDF:** Características como conteo de palabras emocionales ya están implícitamente capturadas por pesos TF-IDF de esos términos. Agregar conteos explícitos introduce colinealidad.
2. **Escala de features:** Las 18 características manuales están en escalas numéricas pequeñas (0-100), mientras TF-IDF produce miles de features con valores [0-1]. Sin normalización adecuada, las features TF-IDF dominan la señal.
3. **Posible sobreajuste:** Las features se diseñaron observando ejemplos del conjunto de entrenamiento, potencialmente capturando patrones idiosincráticos que no generalizan bien.

Este hallazgo negativo es informativo: sugiere que para tareas de clasificación emocional con datasets grandes (400k+ muestras), la representación TF-IDF de 5000 términos ya es

suficientemente expresiva. Feature engineering manual agrega complejidad sin beneficio medible.

VI-E. Implicaciones prácticas

Para profesionales considerando implementar clasificadores emocionales, nuestros resultados sugieren:

- **Prototipado rápido:** Comenzar con TF-IDF + Regresión Logística antes de invertir en infraestructura para deep learning. Este baseline puede ser suficiente para muchas aplicaciones.
- **Trade-off costo-rendimiento:** Si el 2-3 % de mejora en F1 justifica 10-100x más costo computacional depende del caso de uso. Para aplicaciones críticas (moderación de contenido, soporte mental), el gap puede ser significativo. Para análisis exploratorio de sentimientos, el modelo clásico es adecuado.
- **Deployment edge:** Modelos clásicos son viables para deployment en dispositivos móviles o servidores sin GPU, democratizando acceso a tecnología de análisis emocional.
- **Regulación y explicabilidad:** En jurisdicciones con requerimientos de explicabilidad de IA (ej: GDPR en Europa), la interpretabilidad de modelos lineales puede ser ventaja regulatoria sobre cajas negras neuronales.

VII. TRABAJO FUTURO

VII-A. Mejoras algorítmicas

1. Representaciones híbridas: Combinar embeddings pre-entrenados contextuales (ej: BERT embeddings congelados) con TF-IDF podría capturar tanto información semántica profunda como relevancia estadística. Una arquitectura posible:

$$x_{\text{hybrid}} = [TF\text{-}IDF(\text{text}), \text{mean}(BERT(\text{text}))] \quad (5)$$

Esto mantendría la eficiencia de clasificadores lineales mientras aprovecha representaciones contextuales.

2. N-gramas de mayor orden: Explorar n-gramas hasta (1,3) o (1,4) para capturar expresiones idiomáticas complejas (ej: “over the moon happy”, “scared out of my mind”). Requiere balancear dimensionalidad con ganancia informativa.

3. Atención a nivel de palabra: Implementar un mecanismo de atención liviano que aprenda pesos de palabras específicos por tarea, similar a:

$$\alpha_i = \frac{\exp(w \cdot TF\text{-}IDF_i)}{\sum_j \exp(w \cdot TF\text{-}IDF_j)} \quad (6)$$

donde w es un vector de pesos aprendidos. Esto permitiría al modelo enfocarse adaptativamente en términos relevantes.

VII-B. Extensión a español y multilingüismo

El proyecto actual se limita a inglés. Extensiones importantes incluyen:

- 1. Corpus en español:** Crear/recopilar un dataset equivalente en español mediante:
 - Traducción automática del Emotion Dataset con post-edición humana

- Anotación manual de tweets en español usando plataformas de crowdsourcing
- Explorar corpus existentes como TASS (Taller de Análisis Semántico en la SEPLN)

2. Modelos multilingües: Entrenar clasificadores que funcionen en múltiples idiomas simultáneamente mediante:

- Embeddings multilingües alineados (ej: MUSE, LASER)
- Transfer learning desde inglés a español
- Zero-shot cross-lingual classification

3. Análisis contrastivo: Estudiar cómo diferencias culturales en expresión emocional afectan rendimiento del modelo. ¿Los hispanohablantes expresan emociones de manera distinguible de anglohablantes?

VII-C. Aplicaciones en producción

1. Integración con APIs de mensajería: Implementar conectores para WhatsApp Business API, Telegram Bot API, y Slack para análisis emocional en tiempo real de conversaciones. Casos de uso:

- Chatbots empáticos que adaptan tono según emoción detectada
- Sistemas de soporte técnico que priorizan tickets con frustración/ira
- Monitores de salud mental que detectan tristeza persistente

2. Sistema de feedback activo: Implementar aprendizaje continuo donde usuarios pueden corregir clasificaciones incorrectas, permitiendo mejora iterativa del modelo en producción mediante:

- Reentrenamiento periódico con datos corregidos
- Active learning para identificar ejemplos ambiguos que requieren etiquetado

3. Análisis agregado temporal: Desarrollar dashboards que visualicen evolución emocional en comunidades/usuarios a lo largo del tiempo, útil para:

- Monitoreo de salud mental de comunidades en riesgo
- Análisis de satisfacción de clientes post-producto
- Detección de crisis emocionales en redes sociales

VII-D. Investigación metodológica

1. Análisis de sesgo: Investigar si el modelo exhibe sesgos demográficos o culturales:

- ¿Clasifica diferentemente emociones expresadas por hombres vs mujeres?
- ¿Existe sesgo contra dialectos específicos (ej: inglés africano vs británico)?
- ¿Términos asociados con minorías reciben clasificación diferente?

2. Interpretabilidad avanzada: Aplicar técnicas de XAI (Explainable AI) como LIME o SHAP para entender decisiones del modelo a nivel de instancia individual. Esto podría revelar patrones no obvios en cómo el modelo procesa lenguaje emocional.

3. Clasificación jerárquica: Explorar taxonomías emocionales multi-nivel donde emociones se organizan jerárquicamente:

- Nivel 1: Valencia (positivo/negativo/neutral)
- Nivel 2: Emoción básica (joy, sadness, anger...)
- Nivel 3: Emociones finas (euphoric, content, amused...)

Esto permitiría predicciones a diferentes niveles de granularidad según confianza.

VIII. CONCLUSIONES

Este trabajo presenta un análisis de técnicas clásicas de PLN para clasificación de emociones en texto, demostrando que métodos tradicionales pueden alcanzar rendimiento competitivo cuando se optimizan cuidadosamente. Los hallazgos principales son:

1. Viabilidad de métodos clásicos: TF-IDF + Regresión Logística con balanceo de clases alcanzó 90.8 % en F1-Score y 88.8 % precision macro, superando el objetivo de 80 % y posicionándose solo 2-3 % por debajo de modelos Transformer de última generación. Esto demuestra que la representación TF-IDF de 5000 términos captura información suficiente para clasificación emocional efectiva en textos cortos.

2. Importancia del balanceo de clases: SMOTE combinado con `class_weight='balanced'` fue importante para mejorar recall de clases minoritarias. El trade-off precision-recall resultante sugiere límites fundamentales del sobremuestreo sintético. Estrategias futuras deberían explorar recolección de más datos reales de clases minoritarias en lugar de depender exclusivamente de augmentación sintética.

3. Optimización de hiperparámetros: GridSearchCV produjo mejoras modestas (+0.15 % promedio), indicando que los parámetros por defecto de scikit-learn ya son razonables para esta tarea. La mayor parte de la mejora provino de la elección de representación (TF-IDF) y balanceo de clases, no de ajuste fino de hiperparámetros.

4. Limitaciones del feature engineering: Las 18 características adicionales diseñadas manualmente no mejoraron rendimiento del ensemble, sugiriendo que para datasets grandes (400k+ muestras), la representación TF-IDF es suficientemente expresiva. Este hallazgo ahorra esfuerzo futuro en feature engineering para tareas similares.

5. Aplicabilidad práctica validada: El prototipo web funcional demuestra que clasificadores clásicos son viables para deployment en producción con inferencia < 50ms y sin requerir hardware especializado. Esto facilita acceso a tecnología de análisis emocional, particularmente para startups y organizaciones con recursos computacionales limitados.

6. Trade-off interpretabilidad-rendimiento: El pequeño gap de rendimiento (2-3 %) respecto a modelos de deep learning puede justificarse en escenarios donde interpretabilidad es prioritaria. Los coeficientes de regresión logística permiten auditar exactamente qué términos influyen en cada decisión, facilitando cumplimiento regulatorio y detección de sesgos.

VIII-A. Reflexión final

En una era dominada por modelos de lenguaje masivos con billones de parámetros, este proyecto reafirma que las técnicas clásicas de machine learning mantienen relevancia práctica. El 90.8 % de F1-Score logrado con un modelo de <150MB que entrena en minutos en CPU demuestra que la complejidad no siempre equivale a efectividad. Para muchas aplicaciones reales, especialmente en contextos con recursos limitados o requerimientos de interpretabilidad, los enfoques tradicionales optimizados pueden ser la mejor opción.

El futuro de NLP aplicado probablemente no será una elección binaria entre métodos clásicos y deep learning, sino una integración inteligente de ambos: usando representaciones profundas donde aporten valor único, y clasificadores lineales donde sean suficientes. Este proyecto contribuye a esa visión pragmática.

REFERENCIAS

- [1] P. Ekman, "An argument for basic emotions," *Cognition & Emotion*, vol. 6, no. 3-4, pp. 169-200, 1992.
- [2] R. Pekrun, T. Goetz, W. Titz, and R. P. Perry, "Academic emotions in students' self-regulated learning and achievement: A program of qualitative and quantitative research," *Educational Psychologist*, vol. 37, no. 2, pp. 91-105, 2002.
- [3] E. Cambria, S. Poria, A. Gelbukh, and M. Thelwall, "Sentiment analysis is a big suitcase," *IEEE Intelligent Systems*, vol. 32, no. 6, pp. 74-80, 2017.
- [4] K. Sailunaz and R. Alhaji, "Emotion and sentiment analysis from Twitter text," *Journal of Computational Science*, vol. 36, p. 101003, 2019.
- [5] I. Perikos and I. Hatzilygeroudis, "Recognizing emotions in text using ensemble of classifiers," *Engineering Applications of Artificial Intelligence*, vol. 51, pp. 191-201, 2016.
- [6] S. Wang and C. D. Manning, "Baselines and bigrams: Simple, good sentiment and topic classification," in *Proc. 50th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2012, pp. 90-94.
- [7] L. A. B. Bostan and R. Klinger, "An analysis of annotated corpora for emotion classification in text," in *Proc. 27th International Conference on Computational Linguistics (COLING)*, 2020, pp. 2104-2119.
- [8] K. R. Scherer and H. G. Wallbott, "The ISEAR questionnaire and codebook," Geneva Emotion Research Group, Tech. Rep., 1997.
- [9] C. Strapparava and R. Mihalcea, "SemEval-2007 Task 14: Affective text," in *Proc. 4th International Workshop on Semantic Evaluations (SemEval)*, 2007, pp. 70-74.
- [10] L. A. B. Bostan and R. Klinger, "A survey on distant supervision for relation extraction," *ACM Computing Surveys*, vol. 51, no. 5, pp. 1-35, 2018.
- [11] E. Saravia, H. C. T. Liu, Y. H. Huang, J. Wu, and Y. S. Chen, "CARER: Contextualized affect representations for emotion recognition," in *Proc. 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 3687-3697.
- [12] "Emotion dataset card," Hugging Face. [Online]. Available: <https://huggingface.co/datasets/emotion>. Accessed: Nov. 2024.
- [13] D. Demszky, D. Movshovitz-Attias, J. Ko, A. Cowen, G. Nemade, and S. Ravi, "GoEmotions: A dataset of fine-grained emotions," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 4040-4054.
- [14] F. A. Acheampong, H. Nunoo-Mensah, and W. Chen, "Transformer models for text-based emotion detection: A review of BERT-based approaches," *Artificial Intelligence Review*, vol. 54, pp. 5789-5829, 2021.
- [15] L. Plaza, A. Díaz, and P. Gervás, "A semantic graph-based approach to biomedical summarisation," *Artificial Intelligence in Medicine*, vol. 53, no. 1, pp. 1-14, 2011.
- [16] F. N. Ribeiro, M. Araújo, P. Gonçalves, M. A. Gonçalves, and F. Benevenuto, "SentiBench - A benchmark comparison of state-of-the-practice sentiment analysis methods," *EPJ Data Science*, vol. 5, no. 1, pp. 1-29, 2016.